Summary:

My code builds and trains a multi-layer perceptron from scratch. It loads the data, defines the activation functions, performs forward and backward passes, and updates weights using gradient descent. I ran the process for multiple epochs while tracking the accuracy of each test, evaluating the model's performance across different configurations. The results show that the number of hidden layers and neurons per hidden layer does not significantly affect the accuracy. However, the learning rate has a notable impact on the score, with smaller learning rates possibly helping the model converge better.

Results:

Summary of Results:

Base Accuracy (Untrained): Accuracy = 40.36%

2 hidden layers, 4 neurons each: Accuracy = 74.89%

3 hidden layers, 4 neurons each: Accuracy = 74.89%

4 hidden layers, 4 neurons each: Accuracy = 74.89%

2 hidden layers, 4 neurons each: Accuracy = 74.89%

2 hidden layers, 8 neurons each: Accuracy = 74.89%

2 hidden layers, 16 neurons each: Accuracy = 74.89%

2 hidden layers, 4 neurons each, learning rate 0.125: Accuracy = 59.64%

2 hidden layers, 4 neurons each, learning rate 0.0625: Accuracy = 74.89%

2 hidden layers, 4 neurons each, learning rate 0.03125: Accuracy = 74.89%